Part III Report

1. Baseline Test: GET vs. POST (1 Worker, 50 Users)

Using the StandardHttpUser, we established a performance baseline with a 3:1 ratio of GET to POST requests.

- Total RPS: 32.4
- GET Average Latency: 7.84 ms (Average size: 124,785 bytes)
- POST Average Latency: 3.40 ms (Average size: 95 bytes)
- Failures: 0%

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET Albums	6180	0	7	14	21	7.84	1	100	124785.31	25.5	0
POST	POST Album	2049	0	3	6	9	3.4	1	34	95	6.9	0
	Aggregated	8229	0	6	14	19	6.74	1	100	93737.74	32.4	0

```
CONTAINER ID   NAME              CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
BLOCK I/O   PIDS
4d7f48b15420   part3-worker-1    18.70%    48.1MiB / 7.653GiB    0.61%     1.16GB / 13MB
0B / 0B     3
669d40ffc949   part3-master-1    0.05%     35.78MiB / 7.653GiB   0.46%     380kB / 483kB
0B / 0B     3
```

Observation & Tradeoff Analysis:

In our implementation, POST requests were significantly faster than GET requests.

- According to the statistics, GET /albums returns a payload of ~122 KB, whereas POST returns only 95 bytes. The time spent on JSON serialization and transmitting 122KB over the network far outweighs the cost of the server-side write operation.
- Real-world Tradeoffs: In a production environment, GET operations are typically more

frequent (Read-Heavy). If our album collection grows to thousands of items, the GET latency will degrade further.

- Data Structure Impact: We used a Hashmap protected by a Mutex. While a Hashmap provides $O(1)$ lookup, the Mutex locks the entire map during writes. In a real-world scenario with high read-concurrency, switching to a sync.RWMutex (allowing multiple concurrent readers) would be more efficient to prevent the "Read-locking" from being a bottleneck.

2. Scaling & Amdahl's Law (4 Workers)

We increased the worker count to 4 to observe scaling behavior.

- Total RPS (1 Worker): 32.4
- Total RPS (4 Workers): 33.3
- Total Speedup: ~1.03x (Non-linear)

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET Albums	4192	0	5	10	13	5.89	1	27	81022.55	24	0
POST	POST Album	1331	0	3	5	8	3.31	1	22	95	9.3	0
	Aggregated	5523	0	5	10	13	5.26	1	27	61519.64	33.3	0

```
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
  BLOCK I/O   PIDS
4d7f48b15420   part3-worker-1   10.42%    46MiB / 7.653GiB      0.59%     2.64GB / 26.2MB
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
  BLOCK I/O   PIDS
4d7f48b15420   part3-worker-1   10.42%    46MiB / 7.653GiB      0.59%     2.64GB / 26.2MB
  0B / 0B     3
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
  BLOCK I/O   PIDS
4d7f48b15420   part3-worker-1   6.58%     51.31MiB / 7.653GiB   0.65%     2.65GB / 26.2MB
CONTAINER ID   NAME             CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
 BLOCK I/O   PIDS
210ce6d2168b   part3-worker-3   3.16%     41.09MiB / 7.653GiB   0.52%     167MB / 2.5MB
 0B / 0B     3
63a1314aa8ab   part3-worker-4   4.22%     46.54MiB / 7.653GiB   0.59%     157MB / 2.41MB
 0B / 0B     3
bd2d9d50ac4e   part3-worker-2   4.08%     46.12MiB / 7.653GiB   0.59%     155MB / 2.13MB
 0B / 0B     3
4d7f48b15420   part3-worker-1   4.10%     40.55MiB / 7.653GiB   0.52%     168MB / 2.27MB
 0B / 0B     3
669d40ffc949   part3-master-1   0.58%     36.25MiB / 7.653GiB   0.46%     778kB / 643kB
 0B / 0B     3
```

Analysis of Amdahl's Law:

Increasing resources by 400% only yielded a 2.7% increase in throughput.

- The Evidence: docker stats shows that when running 4 workers, the CPU load per worker dropped from 18.70% to ~4.1%. This indicates the workers were idle, waiting for the server.
- The Bottleneck: This is a classic demonstration of Amdahl's Law. The "Serial Fraction" of our system is the Go Server's request handling logic. Specifically, the Global Mutex on the album map ensures that only one goroutine can modify the map at a time. This serialization, combined with the overhead of handling the large GET response on a single network interface, prevents linear scaling.

3. Context Switching: HttpUser vs. FastHttpUser

We compared the standard HttpUser with the C-based FastHttpUser (1 Worker).

- StandardHttpUser CPU Usage: 18.70%
- FastHttpUser CPU Usage: 14.39%
- RPS: Remained stable at ~33.4

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	FAST GET Albums	6357	0	6	13	19	6.65	1	92	125234.06	24.5	0
POST	FAST POST Album	2045	0	2	3	9	1.87	0	25	96	8.9	0
	Aggregated	8402	0	5	12	19	5.48	0	92	94776.15	33.4	0

```
CONTAINER ID   NAME              CPU %     MEM USAGE / LIMIT     MEM %     NET I/O
BLOCK I/O   PIDS
ec84cf792f5c   part3-worker-1   14.39%    37.48MiB / 7.653GiB   0.48%     726MB / 9.1MB
0B / 0B     3
7ea0ed03a74a   part3-master-1   0.20%     36.36MiB / 7.653GiB   0.46%     304kB / 310kB
0B / 0B     3
```

Analysis of Context Switching:

FastHttpUser resulted in a 23% reduction in CPU consumption on the load generator.

- Mechanism: HttpUser uses Python’s requests library, which is synchronous and relies on heavier threading/greenlet management. FastHttpUser utilizes a C-based geventhttpclient.
- Result: By reducing the overhead of context switching and memory allocation within Python, FastHttpUser can simulate much higher concurrency with fewer local resources.

This is crucial for high-throughput testing where the testing tool itself might otherwise become the bottleneck.

4. Conclusion

The load tests reveal that our server is currently bound by its serial components (Mutex and Serialization) rather than the load generation capacity. To improve performance, we should look into finer-grained locking (e.g., sharded maps) or improving the efficiency of the GET response serialization.